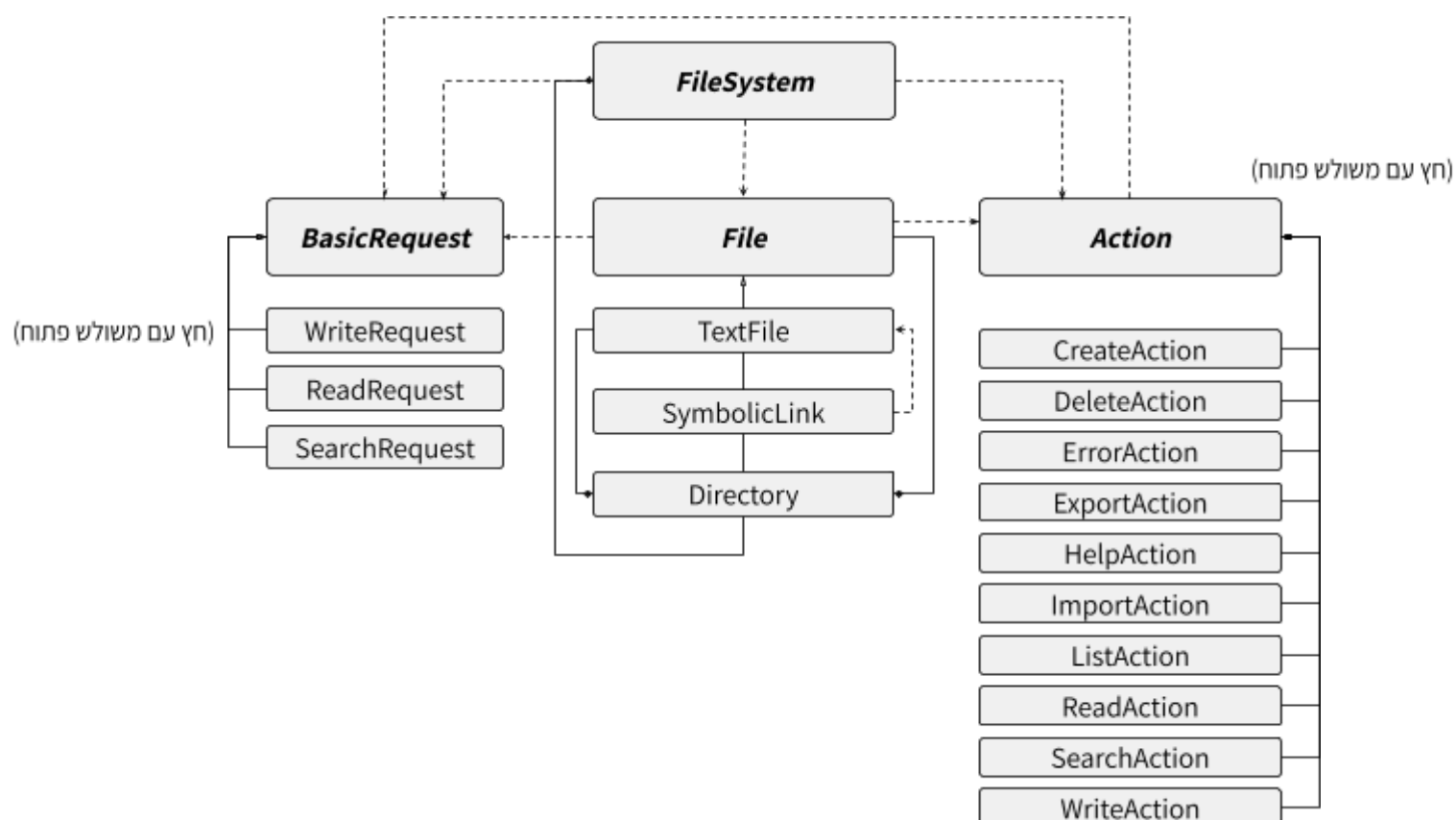


-8

- 8.1



-8.2

תבניות התכן שבשימוש:

1. תבנית Singleton-

מכיוון שאנחנו רוצים שתתקיים מערכת קבצים אחת ויחידה לכל אורך הריצה של התכנית השתמשנו בתבנית Singleton עבור FileSystem. הדבר בא לידי ביטוי במספר דרכים. ראשית, הגדרנו את הבנאי FileSystem () תחת private דבר שמונע מכל גורם חיצוני ליצור מופע חדש באמצעות הבנאי. כמו כן, הוספת ה "delete =" לבנאי ההעתקה וההשמה חוסמת את האפשרות לשכפל את המערכת. בנוסף, השימוש במשתנה סטטי static FileSystem instance במתודה getInstance מבטיחה לנו אתחול של אובייקט יחיד של FileSystem שיוצר רק בקריאה הראשונה שלו.

2. תבנית Composite-

במחלקת הקבצים אנחנו משתמשים בירושה שתאפשר לנו ליצור עץ קבצים שיכול להכיל בתוכו תיקיות לעומק לא מוגבל על פי העקרונות הבאים -

- רכיב Component - המחלקה האבסטרקטית, מחלקת File
- רכיבי העלים Leaf - מחלקות SymbolicLink & TextFile אשר יורשות מFile אך לא מכילות ילדים.
- רכיב Composite - מחלקת Directory שיוורשת מFile אך מחזיקה במצביעים לילדים מטיפוס File.

3. תבנית Command-

יצרנו מחלקה אבסטרקטית Action אשר ממנה יורשות מחלקות נוספות ממוקדות פעולה אשר אורזות לתוך אובייקט את כל הפרמטרים הנדרשים לטובת ביצוע אותה פעולה. בכך אפשרנו לעצמנו לשמור את היסטוריית הפעולות שהתרחשו בהצלחה (ואם נרצה לעשות undo בעתיד נוכל באמצעות שמירת האובייקט על פי סדר ויצירת פעולת חזרה), אפשרנו גמישות בהפעלת הפעולה (היא לא מופעלת מיד אלא שמורה כאובייקט ונוכל להפעיל אותה בכל מקום בו נחליט ללא צורך בשינוי דרמטי בקוד) וכן קריאות וסדר בקוד (היא מופרדת למחלקה נפרדת ולא מסרבלת את הקוד, מחזירה טיפוס מסוג פעולה ומבצעת במקטע בו הוגדר).

4. תבנית Factory-

במקום לכתוב מתודות פעולה ובקשה שמותנות כל אחת בסוג הקובץ ובכך ליצור קוד סבוך של if else יצרנו שלוש מתודות וירטואליות טהורות File כך שהמחלקות היורשות ממשות את המתודות ומחזירות את ה Action הרלוונטי לכל אחת.

-8.3

ראשית נוסף מחלקה `compressedFile` אשר יורשת פומבית מ-`File`. כך היא תירש מ-`File` את השדות של שם הקובץ ותיקיית האב כנדרש מכל סוג של קובץ. בנוסף, נממש לה את המתודות הוירטואליות שירשה מ-`File` גם כן (`Create***Action`), כך כמו יתר הקבצים היא תחזיר את ה-`Action` הרלוונטי אליה או שגיאות על פעולות שאינן מתאפשרות לבצע על קובץ דחוס על פי מה שיוגדר. במקומות בקוד בהם אנו מעבירים אובייקט קובץ כללי נוסף את האפשרות לקבל גם קובץ דחוס. אם קישור סימבולי יכול להצביע על קובץ מסוג דחוס נוסף בקוד של מחלקת `SymbolicLink` בדיקה שבדקת האם הקישור מצביע על אובייקט זה (בנוסף לקובץ הטקסט) אך מבלי לשנות את הבדיקות והקוד הכללי שקיים עד כה (נעשה זאת תחת המתודה של `isBrokenLink` כך שלא נצטרך לשנות את המחלקה כלל מלבד המתודה הנ"ל).

מעבר לשינויים אלו, לא נצטרך לשנות את מבנה הקוד בזכות תבניות התכן בהן השתמשנו. מבנה וזרימת הקוד נותרו זהים בזכות הכללת הפעולות לכל סוג קובץ אשר יתקבל והתאמות נקודתיות לכל סוג ספציפי.

-8.4

ניתן לממש את `copy` בתכנית שכתבנו. מכיוון שהפעולה `copy` יוצרת קובץ וכותבת לתוכו אנחנו נחשיב את הפעולה תחת פעולות הכתיבה (`WriteAction`) ונחסוך ביצירת מתודה וירטואלית חדשה.

נגדיר מחלקה `CopyAction` יורשת מ `Action`. כמו כן ניצור `handleCopy` שיטפל במקרה זה וכן נוסיף פקודה חדשה `commandType`.

נגדיר את הפעולה כך שתקבל את הנתיב של הקובץ שברצוננו להעתיק ואת הנתיב של התיקייה אליה נרצה לבצע העתקה (אלו שני התנאים המינימליים, כאשר ניתן לקבוע שאין צורך בנתיב יעד ואז הקובץ יועתק לאותה תיקיית אב. ניתן להגדיר כי נרצה לקבל גם שם עבור הקובץ החדש או שהשם יוגדר כחלק מהפעולה להיות השם של הקובץ המקורי בתוספת מספר הפעמים שהועתק (`file.1`)). הפעולה תחפש רקורסיבית את הנתיב של הקובץ המקורי ותבצע העתקה על פי סוג הקובץ -

אם קובץ טקסט, הפעולה תיצור קובץ טקסט חדש (באמצעות `addChild`) תחת התיקייה המבוקשת (שסופקה על ידי מחרוזת של הנתיב הרצוי) ותעתיק אל הקובץ החדש את התוכן המקורי.

אם תיקייה, ניצור תיקייה חדשה ונעתיק אליה את הילדים של התיקייה המקורית (למעשה נבצע שכפול גם עבור הילדים וכן הלאה).

אם קישור סימבולי, ניצור אובייקט קישור חדש ונעתיק אליו את הנתיב המקורי (מעתיקים את הקישור בלבד, לא את קובץ הטקסט עליו הוא מצביע). אימפלמנטציה זו עושה שימוש בתבנית ה `Factory` שלנו בכך שרק נצטרך ליצור את הרכיבים המתאימים ומטפלי הפעולה החדשה ללא עריכת קוד של המטפלים והפעולות הקיימות!